Lesson 10: Play It Again

GameMaker

VIDEO GAME CREATION

This week: when the round ends, the game actually stops - and pressing R brings it back to a genuinely fresh start.

Open your own [YourName]Game project from Lesson 9 - don't start a new one.

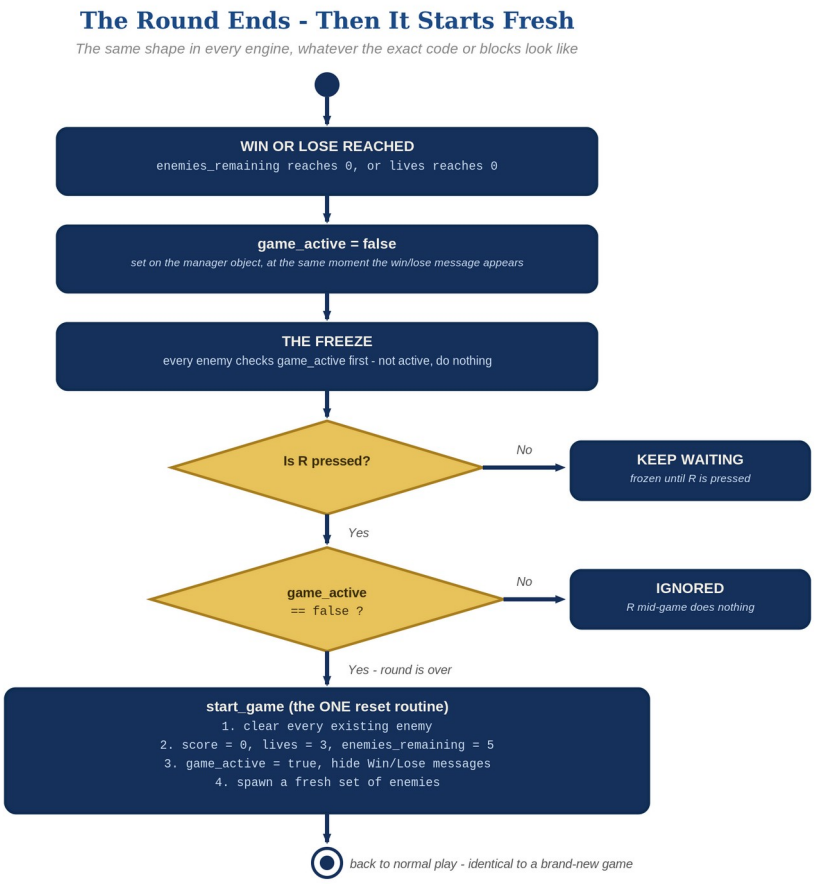
Tip: no new assets this week - this is a logic lesson, and it reuses your very first setup code from Lesson 5.

Setup

1. Open your [YourName]Game project from Lesson 9.
2. Create a new Script resource named scr_start_game - this is GameMaker's way of defining a reusable function, similar to what you'll see in several other engines this week.

Level 1 - Minimal (MVP)

When the round ends (win or lose), the game freezes - score and lives stop changing, and further enemy hits/escapes do nothing. Pressing R resets everything and starts a new round.



When the round ends, everything freezes - and R, pressed only while the round is over, runs the one reset routine. The same shape in every engine.

1. In scr_start_game, write a function start_game() that destroys all existing obj_enemy instances, resets obj_gamemanager's __dnd_score, __dnd_lives, enemies_remaining, has_won, has_lost, and game_active, then spawns a fresh set of enemies (code below).
2. Change obj_spawner's Create event to just call start_game();.
3. On obj_gamemanager's Create event, add game_active = true; alongside your other shared game state.
4. On obj_enemy's Step event, add "if (!obj_gamemanager.game_active) exit;" as the very first line - this freezes movement once the round ends.
5. In obj_bullet's Collision event and obj_enemy's Step event's escape branch, when setting has_won/has_lost, also set obj_gamemanager.game_active = false, and update the has_won/has_lost draw_text() strings in obj_gamemanager's Draw GUI event to include the restart hint.
6. Add a Step event to obj_gamemanager that checks keyboard_check_pressed(ord("R")) and calls start_game() if !game_active.
7. Press F5 (Run), play to a win or loss, confirm movement freezes, then press R and confirm everything resets and play resumes.

8. Keep your sounds: if you added sfx_win, sfx_lifelost and sfx_gameover in Lessons 6 and 8 (Level 3), they stay. The code below shows their audio_play_sound lines as plain, unhighlighted lines. In GML Visual, keep your Play Sound blocks where they already are. If you skipped that goal, leave those lines out.

Highlighted lines are new this week; everything else is code you already have.

```
// scr_start_game (Script resource - new):
function start_game() {
    with (obj_enemy) {
        instance_destroy();
    }
    obj_gamemanager.__dnd_score = 0;
    obj_gamemanager.__dnd_lives = 3;
    obj_gamemanager.enemies_remaining = 5;
    obj_gamemanager.has_won = false;
    obj_gamemanager.has_lost = false;
    obj_gamemanager.game_active = true;
    for (var enemy_count = 0; enemy_count < 5; enemy_count++) {
        instance_create_layer(80 + enemy_count * 96, 80, "Instances",
obj_enemy);
    }
}

// obj_spawner's Create event (revised - the spawn loop moved into
start_game()):
start_game();

// obj_gamemanager's Create event:
enemies_remaining = 5;
has_won = false;
__dnd_lives = 3;
has_lost = false;
__dnd_score = 0;
game_active = true;

// obj_gamemanager's Step event (new):
if (!game_active && keyboard_check_pressed(ord("R"))) {
    start_game();
}

// obj_enemy's Step event (revised):
if (!obj_gamemanager.game_active) exit;
y += enemy_speed;
if (y > room_height) {
    obj_gamemanager.__dnd_lives -= 1;
    if (obj_gamemanager.__dnd_lives <= 0) {
        obj_gamemanager.has_lost = true;
        obj_gamemanager.game_active = false;
        audio_play_sound(sfx_gameover, 1, false);  // your Lesson 8 Level
3 sound
    } else {
        audio_play_sound(sfx_lifelost, 1, false);  // your Lesson 8 Level
3 sound
    }
    instance_destroy();
}
```

```
// obj_bullet's Collision event with obj_enemy (revised):
audio_play_sound(sfx_hit, 1, false);
obj_gamemanager.enemies_remaining -= 1;
obj_gamemanager.__dnd_score += 10;
if (obj_gamemanager.enemies_remaining <= 0) {
    obj_gamemanager.has_won = true;
    obj_gamemanager.game_active = false;
    audio_play_sound(sfx_win, 1, false);  // your Lesson 6 Level 3 sound
}
instance_destroy(other);
instance_destroy(self);

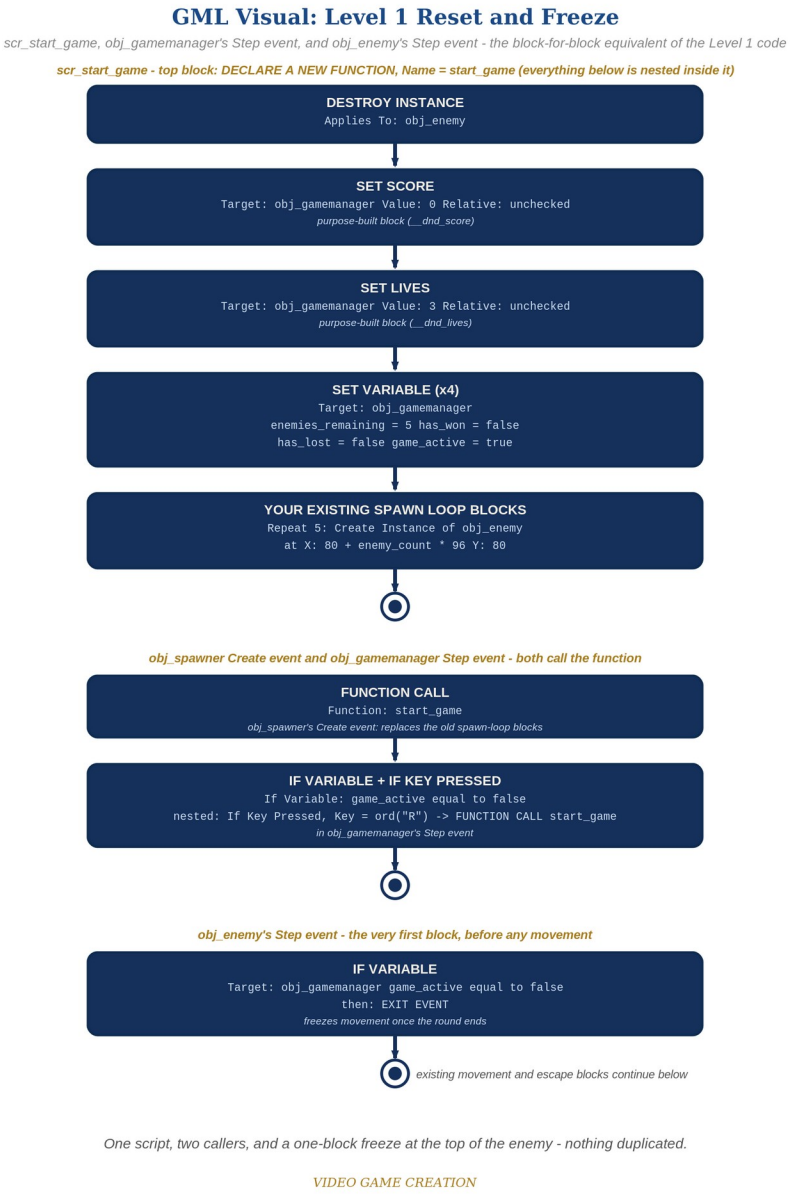
// obj_gamemanager's Draw GUI event (revised):
draw_text(20, 20, "Score: " + string(__dnd_score));
draw_text(20, 50, "Lives: " + string(__dnd_lives));
if (has_won) {
    draw_text(400, 50, "You Win! Press R to Restart");
}
if (has_lost) {
    draw_text(400, 90, "Game Over - Press R to Restart");
}
```

Alternate Path: GML Visual (no typing)

The same reusable-reset pattern works with drag-and-drop blocks instead of typed code. Every Score, Lives, and Set Variable block that changes shared state needs its Target set to obj_gamemanager, exactly as in Lessons 8 and 9.

1. Create a new Script resource named scr_start_game. Its top-level block must be "Declare a New Function" with Name = start_game, and EVERY block below goes INSIDE that function block, including the whole Repeat loop: (1) an "Instance - Destroy Instance" block with Applies To set to obj_enemy; (2) a "Set Score" block, Target obj_gamemanager, Value 0, Relative unchecked; (3) a "Set Lives" block, Target obj_gamemanager, Value 3, Relative unchecked; (4) four "Set Variable" blocks with Target obj_gamemanager: enemies_remaining = 5, has_won = false, has_lost = false, and game_active = true; (5) your existing spawn-loop blocks (Repeat 5 with Create Instance inside). Check the connectors: any block that is not nested inside the function runs once when the game starts, before any room exists, and the Create Instance block will fail with a "layer does not exist" error.
2. On obj_spawner's Create event, replace the spawn-loop blocks with a single "Function Call" block, Function = start_game (leave Target empty and Temp unchecked). Use Function Call, not the Execute Script block: Execute Script runs the script resource itself, which only declares the function and creates no enemies.
3. On obj_gamemanager's Create event, add a "Set Variable" block: game_active = true.
4. On obj_gamemanager's Step event, add an "If Variable" block (the one with a variable name, an operation, and a value - not the true/false block) checking game_active equal to false. Nest an "If Key Pressed" block inside it: click the selector to the right of the Key field and choose Letters, then R, which fills the Key field with ord("R"). Inside that, nest a "Function Call" block, Function = start_game.
5. On obj_enemy's Step event, add an "If Variable" block at the very top: Target obj_gamemanager, game_active equal to false, with an "Exit Event" block nested INSIDE it (check the connector, as above). The movement blocks go after the If block,

not inside it. In both places where has_won or has_lost is set to true (obj_bullet's Collision event and obj_enemy's escape branch), add a "Set Variable" block right after it with Target obj_gamemanager: game_active = false.



scr_start_game, obj_gamemanager's Step event, and obj_enemy's Step event - the block-for-block equivalent of the Level 1 code.

Before You Test: Audit Your Game State

Game state is everything your game remembers right now: score, lives, positions, costumes and colors, animations, sounds, and messages on screen. Until this lesson, your game started once and ended once, so nothing ever had to be put back. A restart means ALL of it must go back to how it was at the start - including anything extra you added that this handout doesn't mention.

1. Find EVERY object's Create event. A Create event runs only when that instance is first created, so objects that already exist when the game restarts (your player,

obj_gamemanager) will NOT run it again. Anything it sets must also be set in start_game(), or in a reset you call from it.

2. Look at everything that happens at game over: image_blend or image_alpha changes, sprite_index or image_speed changes, an animation or Alarm you started, draw calls, sounds, hidden (visible = false) or moved instances.
3. For each, add the opposite to start_game() (or the Visual blocks it calls): restore the sprite, image_blend, image_alpha and image_speed, stop the Alarm, stop the sound, set visible back, and put the player at its starting x and y. In Visual, use Set Variable / Set Sprite style blocks with the Target set to the right object.
4. Check Room Start events and Alarm events as well as Create events: anything that runs only once at launch needs a reset on restart.

Rule of thumb: if it changed when the round ended (or at any point during play), something has to change it back when the round restarts.

Level 2 - Complete (Alpha/Beta)

A restart is a genuinely clean slate - no leftover enemies, no stale score/lives, and it works correctly no matter how many times in a row you play.

- Play to a win, confirm movement freezes, then press R and confirm __dnd_score/__dnd_lives/enemies_remaining are exactly back to 0/3/5 with no leftover obj_enemy instances.
- Repeat for a loss - same clean reset.
- Press R while the game is still active and confirm NOTHING happens.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- Fix a real bug! We laid the groundwork for this bug all the way back in Lesson 6 when we defined our win scenario. Then in Lesson 8 we defined our lose scenario. By thinking in terms of win and lose separately, rather than in terms of wave over and game over, we created a bug. Find it and resolve it in the manner you, the Designer, think best. (Hint: play a round where you destroy some enemies and let the others escape, and watch what happens to the round, and to your R key.)
- Visual: add a short screen-flash or fade using draw_set_alpha() in Draw GUI when the game freezes.
- Structure: notice start_game() is now the single place that sets up a fresh game - called both from obj_spawner's Create event and from its own restart check in Step. No setup code is duplicated anywhere.

If Something's Wrong

- Restart does nothing when you press R: confirm keyboard_check_pressed(ord("R")) is inside an event that actually runs every frame (obj_gamemanager's Step), and that ! game_active is true at that point.
- Restart leaves old enemies on screen: confirm "with (obj_enemy) { instance_destroy(); }" runs BEFORE the spawn loop inside start_game(), not after.
- Enemies still move after game-over: confirm "if (!obj_gamemanager.game_active) exit;" is the very FIRST line inside obj_enemy's Step event, before y += enemy_speed.

- Score or lives don't reset: confirm start_game() sets obj_gamemanager.__dnd_score and obj_gamemanager.__dnd_lives (with the double leading underscore) - the old built-in score and lives are deprecated and not what the HUD reads.

Want More?

Student Resource List > GameMaker section, or gamemaker.io/en/tutorials